SupportIQ Architecture — Presentation Script

Read this word-for-word when presenting to AI architects or technical decision makers

---

> ***HOW TO USE THIS SCRIPT***
>
> - `SAY:` → *Read these words out loud, exactly as written*
> - `[ACTION]` → *Do this on your screen before speaking the next line*
> - `[POINT TO]` → *Move your mouse cursor / finger to this section in the document*
> - `[PAUSE]` → *Stop talking for 2–3 seconds. Let them read. Silence is powerful.*
> - *Everything in* `( )` *is a note for you — do NOT read it out loud*

BEFORE YOU START — Setup Checklist

> *(Do this before the session begins)*

- ☐ Open `ARCHITECTURE.pdf` on your screen — share screen or hand to audience
- ☐ Scroll to the very top — Section 1 "System Overview" should be visible
- ☐ This script open on your phone or a second monitor
- ☐ App running at `http://localhost:8080` if you want to show live endpoints

OPENING — Setting the Context

> *(Audience has just seen the demo, or you are about to start a technical deep-dive)*

SAY:

> *"What you just saw was the output. I want to show you the architecture that produced it — because the interesting decisions are underneath the demo."*

[ACTION: Open `ARCHITECTURE.pdf` — share your screen. Make sure it is at Section 1.]

SAY:

> *"This document covers the full technical architecture — the Spring Boot layer, the AI integration pattern, the error boundary design, and the multi-agent development system that built the platform. I will walk you through the decisions that matter and why they were made that way."*

ACT 1 — System Overview & Design Philosophy (~2 minutes)

[ACTION: Point to Section 1 — System Overview]

[POINT TO: the sentence "AI should be a callable component, not a rewrite"]

SAY:

> *"This is the thesis of the entire platform. AI should be a callable component, not a rewrite. The entire AI integration is a thin synchronous gateway over a standard HTTP client. No streaming. No embeddings. No orchestration framework."*

[PAUSE — 2 seconds.]

SAY:

> *"Before anyone asks — yes, this was a deliberate constraint. The goal was to demonstrate that LLM integration is fundamentally an HTTP call with structured input and output. Not a framework problem. Not a Python problem. A Java Spring Boot problem, solved with tools a Java engineer already knows."*

[POINT TO: the two-layer paragraph near the bottom of Section 1]

SAY:

> *"And there is a second important point in this section. This platform was itself built using a multi-agent AI development system. I will cover that in detail in Section 9. For now, just hold the distinction: multi-agent AI was used in the development pipeline, not in the runtime. The platform at runtime makes single synchronous calls. The AI that built it was orchestrated."*

[PAUSE — 3 seconds.]

ACT 2 — Technology Stack (~2 minutes)

[ACTION: Scroll to Section 2 — Technology Stack table]

SAY:

> *"Look at the tech stack. Java 17, Spring Boot 4.1, Spring Data JPA, H2 in-memory, Jackson, Lombok. A completely standard enterprise Java stack."*

[POINT TO: the AI proxy rows — EPAM DIAL and gpt-5-mini]

SAY:

> *"The only non-standard entries are the AI proxy rows. EPAM DIAL at ai-proxy.lab.epam.com is our internal enterprise AI gateway. It exposes an OpenAI-compatible REST interface. The model is GPT-5-mini-2025-08-07, running behind the DIAL proxy. Requires EPAM VPN."*

[POINT TO: the last three rows — SDLC rows highlighted in bold]

SAY:

> *"And then the three SDLC rows at the bottom. Claude Code for orchestration, Claude Sonnet 4.6 as the agent model, JavaScript workflow scripts. These are development tooling, not runtime components. They built the platform — they do not run inside it."*

[POINT TO: the key constraint note below the table]

SAY:

> *"This line is the most important line in the tech stack section. No LangChain. No Spring AI. No Anthropic SDK. Zero AI-specific dependencies. The entire AI integration uses only spring-boot-starter-webmvc — which includes the RestClient. If you are an enterprise Java architect, that means you can adopt this pattern today without any procurement process."*

[PAUSE — 3 seconds.]

ACT 3 — AI Integration Architecture (~4 minutes)

(This is the core technical section — slow down here)

[ACTION: Scroll to Section 4 — AI Integration Architecture]

[POINT TO: Section 4.1 — the call chain diagram]

SAY:

> *"This is the AI integration architecture. Five components in a straight chain. Let me walk you through each one."*

SAY:

> *"HTTP request arrives at SupportAiController. It validates input and delegates — nothing else. The controller never touches business logic."*

[POINT TO: SupportAiService in the diagram]

SAY:

> *"SupportAiService owns the AI logic. It holds all four system prompt constants. It builds the context string from database data. It orchestrates the call — but it never touches HTTP directly."*

[POINT TO: DialGateway in the diagram]

SAY:

> *"DialGateway is the AI boundary. One method: chat(systemPrompt, userMessage). It constructs the OpenAI request body, POSTs to DIAL, and returns the raw string response. It knows nothing about tickets, prompts, or domain objects. Just HTTP in, string out."*

[POINT TO: AiResponseParser in the diagram]

SAY:

> *"AiResponseParser handles the reality that LLMs do not reliably return clean JSON. It handles three output shapes: bare JSON, markdown-fenced JSON, and prose with embedded JSON. Whichever shape the model returns, the parser extracts the JSON, strips the fences, and deserializes into the target DTO using Jackson."*

[POINT TO: AiParsingException → HTTP 502]

SAY:

> *"If parsing fails — if the model returned something that cannot be parsed — AiParsingException is thrown and caught by GlobalExceptionHandler, which returns a 502. Not a 500. I will explain why that distinction matters in a moment."*

[PAUSE — 3 seconds.]

[ACTION: Scroll to Section 4.2 — DialGateway]

[POINT TO: the wire format JSON block]

SAY:

> *"This is the exact wire format sent to DIAL. Two messages: system role with the system prompt, user role with the context. Max 4096 completion tokens. No temperature parameter, no top_p — deliberate decision to use model defaults for reproducible, consistent output."*

[POINT TO: the @Component annotation in the code snippet]

SAY:

> *"One design note: DialGateway is annotated @Component, not @Service. Semantically it is a gateway — an adapter to an external system. @Service implies business logic ownership. @Component is more honest about what this class does."*

[PAUSE — 2 seconds.]

[ACTION: Scroll to Section 4.4 — Prompt Engineering Architecture]

[POINT TO: the 6-step prompt structure list]

SAY:

> *"Every one of the four system prompts follows the same 6-step structure. Role definition. Enum constraints — the exact valid values the model must use. Scoring rules — numeric scales with labeled breakpoints. Escalation logic — explicit boolean trigger conditions. Schema contract — respond only with valid JSON, no markdown, no explanation. And then the exact JSON schema inline."*

[POINT TO: the example schema contract string]

SAY:

> *"Look at this schema contract. It is not generated from the Java DTO. It is hand-written as a string and embedded at the end of the system prompt. The reason: DTOs contain Jackson annotations and validation annotations that pollute schema generation. And more importantly,*

> *the schema string is part of the model instruction — it must be human-readable at the prompt level for prompt debugging. If you generate it automatically, you lose that visibility.*"

[PAUSE — 3 seconds.]

[POINT TO: the sentiment scoring constraint example]

SAY:

> "*And this is how you get consistent output. Not by hoping the model interprets 'angry' correctly — but by providing the full label-to-range mapping: 1 to 2 is VERY_ANGRY, 3 to 4 is ANGRY, 5 to 6 is NEUTRAL. The UI color coding depends on these values being consistent. This is the prompt engineering that makes that possible.*"

[PAUSE — 2 seconds.]

ACT 4 — The Ticket Analysis Full Trace (~3 minutes)

[ACTION: Scroll to Section 4.5 — AI Data Flow — Ticket Analysis]

[POINT TO: the full data flow trace diagram]

SAY:

> "*This is the complete trace for the most important endpoint — POST /tickets/{id}/analyze. I want to walk through it step by step because it shows every architectural decision in practice.*"

[POINT TO: the buildTicketContext step]

SAY:

> "*After fetching the ticket from the database, the service builds a context string — structured plain text. Ticket number, customer name and email, subject, body. This string is what gets sent to the AI as the user message. It is plain text, not JSON. The AI reads it like a human would read a ticket.*"

[POINT TO: the dialGateway.chat() call with the full DIAL URL]

SAY:

> "*One HTTP POST to DIAL. The URL shows the model name, the deployment path, the API version. This is where the actual AI call happens. Everything before was preparation. Everything after is processing the result.*"

[POINT TO: the WRITE-BACK section — the 6 tick.set() lines]

SAY:

> "*And this is the architectural differentiator. The AI call is not read-only. It mutates the entity. Sentiment score, sentiment label, category, priority, escalation flag — all written back to the SupportTicket record. The status changes to ESCALATED if the AI flagged escalation and the ticket is currently OPEN.*"

[PAUSE — 3 seconds.]

SAY:

> "*The consequence: every future read of this ticket from any endpoint reflects the AI-enriched state. The analysis is cumulative and queryable. This is the difference between AI as a one-off tool and AI as a persistent layer of intelligence.*"

[PAUSE — 3 seconds.]

ACT 5 — Dashboard: Hybrid Architecture (~2 minutes)

[ACTION: Scroll to Section 4.6 — Dashboard]

[POINT TO: the two parallel paths in the dashboard flow]

SAY:

> "*The dashboard is the most architecturally interesting endpoint because it combines two data sources. Java stream aggregations — open count, in-progress count, escalation rate, category breakdown — these are computed entirely in Java from the database. No AI involved.*"

[POINT TO: the statsContext string and the dialGateway call]

SAY:

> "*Then — and only then — the computed statistics are assembled into a plain-text context string and sent to the AI. The AI receives numbers, not ticket content. It never sees customer names, email addresses, or ticket bodies.*"

[PAUSE — 2 seconds.]

SAY:

> "*This is a deliberate data minimisation decision. For queue health assessment, the AI only needs aggregate numbers. Sending full ticket content to the AI for a dashboard call would be unnecessary, more expensive, and a GDPR concern. The architecture enforces privacy by design — not by policy.*"

[POINT TO: the final merge in the builder]

SAY:

> "*The response DTO is assembled by merging both sources. Counts and rates from Java. Queue health score, queue status, and recommendations from AI. The client gets one unified response object.*"

[PAUSE — 2 seconds.]

ACT 6 — Error Handling Architecture (~1.5 minutes)

[ACTION: Scroll to Section 8 — Error Handling]

[POINT TO: the exception type table]

SAY:

> "*Single @RestControllerAdvice. Every exception in the system flows through GlobalExceptionHandler. ResourceNotFoundException gives a 404. Validation failure gives a 400. And then the two AI-specific ones.*"

[POINT TO: AiParsingException → 502 and HttpStatusCodeException → 503]

SAY:

> "*This distinction matters. 502 Bad Gateway means DIAL responded but the model output could not be parsed — the upstream AI logic produced something unusable. 503 Service Unavailable means DIAL itself returned an error — infrastructure issue, VPN disconnected, service degraded.*"

SAY:

> "*From an observability standpoint: a spike in 502s means the prompt schema needs tuning. A spike in 503s means check DIAL or the network. The error code tells you where to look. That is the design intent.*"

[POINT TO: the ErrorResponse JSON example]

SAY:

> "*Every error response has the same shape: timestamp, status, error label, message with the root cause, and the path that triggered it. Structured, consistent, loggable.*"

[PAUSE — 2 seconds.]

ACT 7 — Multi-Agent SDLC (~2 minutes)

(Point to Section 9 but stay high-level — detailed script is in MULTI-AGENT-SCRIPT.md)

[ACTION: Scroll to Section 9 — Multi-Agent SDLC Architecture]

[POINT TO: the Critical distinction box at the top of Section 9]

SAY:

> "*Before I go further — the critical distinction at the top of Section 9. Runtime layer: SupportIQ makes single synchronous AI calls per endpoint. SDLC layer: the platform was built using multi-agent workflows where specialist agents communicate through structured data handoffs.*"

[PAUSE — 2 seconds.]

SAY:

> "*Six specialist agents were configured. Architect, reviewer, tester, support analyst, triage analyst, demo guide. Each with a custom system prompt — a precise job description. Each producing typed JSON output that becomes the input for the next agent.*"

[POINT TO: Section 9.3 — the 6-phase feature implementation flow diagram]

SAY:

> "*This is the feature implementation pipeline. Six phases: research, plan, implement, integrate, test, review. Nine agents total. Four phases run in parallel. Every SupportIQ endpoint was built through this pipeline.*"

[POINT TO: the key design principle text]

SAY:

> "*The key design principle: the workflow is the coordinator. The agents are the specialists. The orchestration logic — what runs in parallel, what waits, what data flows forward — is deterministic JavaScript code, not model-driven. The models handle reasoning within their scope. The workflow handles coordination across scopes.*"

[PAUSE — 3 seconds.]

SAY:

> "*If you want the full walkthrough of the multi-agent pipeline — I have a dedicated script for that. This section is the architectural record of how it was built.*"

ACT 8 — Production Considerations (~2 minutes)

[ACTION: Scroll to Section 12 — Scalability & Production Considerations]

SAY:

> "*I want to spend a minute on what is not production-ready and why — because being honest about the gaps is part of the architecture review.*"

[POINT TO: Section 12.1 — Synchronous AI Calls]

SAY:

> "*Every AI call is synchronous and blocking. A single dialGateway.chat() call can take 1 to 5 seconds depending on DIAL load. Under concurrent load, servlet threads are blocked for the duration. The solution path is async service methods with CompletableFuture or moving to WebFlux with reactive WebClient. Not done here because this is a demo architecture.*"

[POINT TO: Section 12.2 — Database]

SAY:

> *"H2 in-memory database. Wiped on restart. The JPA layer requires no code changes for PostgreSQL — only application.properties and pom.xml. That is a one-afternoon swap."*

[POINT TO: Section 12.5 — PII in AI Context]

SAY:

> *"analyzeTicket sends the full ticket body — including customer name and email — to DIAL. For a GDPR-compliant production deployment, add a PiiRedactor component to replace identifiable fields before the AI call. The architecture has a clear place for it — between buildTicketContext and dialGateway.chat."*

[PAUSE — 2 seconds.]

SAY:

> *"These gaps are documented here not as apologies, but as the architectural roadmap for productionisation. You know exactly what to do and in what order."*

ACT 9 — Architecture Decision Records (~2 minutes)

[ACTION: Scroll to Section 13 — Architecture Decision Records]

SAY:

> *"Five ADRs. I will go through them quickly — each one records a decision that could have gone the other way."*

[POINT TO: ADR-001 — RestClient over WebClient]

SAY:

> *"ADR-001: RestClient over WebClient. The service layer uses @Transactional and JPA — inherently blocking. Mixing reactive WebClient with blocking JPA in the same call chain creates thread-starvation risk on Reactor schedulers. Blocking RestClient keeps the threading model uniform. Consistent is better than clever."*

[POINT TO: ADR-002 — No AI SDK]

SAY:

> *"ADR-002: Zero AI-specific dependencies. No Spring AI, no LangChain, no vendor SDK. Demonstrates that LLM integration is an HTTP call problem, not a framework problem. Avoids SDK version lock-in. The consequence: no built-in retry or streaming — AiResponseParser handles what SDKs would handle automatically."*

[POINT TO: ADR-003 — System prompts as static constants]

SAY:

> *"ADR-003: Prompts as private static final String constants in the service class. Not database rows. Not config files. Prompts encode business logic — what counts as HIGH risk, what triggers escalation. They must be version-controlled, reviewed in PRs, testable. Changing a prompt requires a code deploy. That is the point."*

[POINT TO: ADR-004 — Write-back on analyzeTicket]

SAY:

> *"ADR-004: analyzeTicket writes back to the entity. An analysis that is not persisted has no durable effect. The next agent or the next endpoint gets no benefit from AI work that was discarded. Persistence makes the enrichment cumulative and queryable."*

[POINT TO: ADR-005 — 502 not 500]

SAY:

> *"ADR-005: AiParsingException maps to 502, not 500. 502 means the upstream returned something unusable. 500 would imply an internal application bug. The error code should tell you where the failure is. 502 tells you to look at the AI layer, not the application code."*

[PAUSE — 3 seconds.]

CLOSING — The Architecture Takeaway (~1 minute)

[ACTION: Scroll back to Section 4.1 — the thin AI layer call chain diagram]

SAY:

> *"Look at this diagram again. Five components. Each with exactly one job. Controller validates and delegates. Service builds prompts and context. Gateway does HTTP. Parser extracts JSON. Handler maps errors."*

[PAUSE — 2 seconds.]

SAY:

> *"This is the architecture pattern you want to bring to your organisation. Not because it is elegant — though it is — but because any Java engineer can read it, maintain it, and extend it. No AI degree required. No Python rewrite required. No procurement of a new framework."*

[PAUSE — 3 seconds.]

SAY:

> *"LLM integration is an HTTP call with structured input and structured output. If you get that right — the thin layer, the clean boundary, the typed responses — everything else follows."*

[PAUSE — 5 seconds. Let them ask questions first.]

QUESTION & ANSWER — Ready Responses

Q: "Why H2 and not PostgreSQL? Is this production-ready?"

SAY:

> *"Intentional choice for the demo. H2 requires zero configuration and starts with the application. The JPA layer abstracts the database completely — swapping to PostgreSQL is application.properties and pom.xml, not application code. Section 12 in the document covers this. The architecture is production-ready; the database choice is not."*

Q: "How do you handle DIAL outages? There's no retry logic."

SAY:

> *"Correct — no retry in the current implementation. The GlobalExceptionHandler maps DIAL HTTP errors to 503. For production: add @Retryable from spring-retry on dialGateway.chat() with an exponential backoff policy, or add a circuit breaker with Resilience4j. The boundary is clean — all retries would live in DialGateway without touching any other component."*

Q: "The prompts are in the code. What if a prompt needs to change urgently in production?"

SAY:

> *"This is documented as ADR-003. The decision is intentional — prompts encode business logic and should go through the same deployment gate as code. If you need dynamic prompt management, the right place is a PromptService that reads from a database or config server, injected into SupportAiService. The interface does not change. But for this stage, we want prompt changes to be reviewed in PRs, not applied ad hoc."*

Q: "How do you prevent the AI from returning invalid enum values — like a category that doesn't exist?"

SAY:

> *"Two layers. The system prompt explicitly lists the valid enum values and instructs the model to use only those values. Then AiResponseParser deserializes using Jackson, and if the model returns an unrecognized enum value, Jackson throws a deserialization exception, which propagates as AiParsingException and returns a 502. The error response includes the exact parsing failure message. In practice the model respects the schema contract reliably — the 502 path is the safety net, not the first line of defence."*

Q: "Can this pattern scale to multiple AI models — like using different models for different endpoints?"

SAY:

> *"Yes — and the architecture makes it straightforward. DialGateway accepts a model name as a constructor parameter. You could inject two DialGateway beans — one for gpt-5-mini for fast classification tasks, one for a larger model for draft reply generation. The system prompt constants in SupportAiService already isolate per-endpoint AI logic. The change is localised to AiConfig and DialGateway wiring — no business logic changes."*

Q: "Why not use Spring AI? It handles a lot of this boilerplate."

SAY:

> *"The same question is in ADR-002. Spring AI is a good framework. The choice here was not to use it specifically to demonstrate that the underlying pattern is simple enough to not need a framework. If you understand how the HTTP call works, what the wire format is, how the response is parsed — you can adopt Spring AI with full confidence. If you start with the framework and something breaks, you are debugging inside the abstraction. We started without it deliberately, for that reason."*

FINAL NOTES

- **Know Section 4 (AI Integration)** cold — architects will spend the most time there
- **ADRs are your strongest credibility moment** — they show intentional trade-off thinking, not just "we used Spring Boot"
- **Section 12 (Production Gaps)** should be raised by you, not by them — proactive honesty is more credible than being caught
- **If they ask to see live code** — open `SupportAiService.java` and show the prompt constants. The code is clean and readable directly.
- **The multi-agent SDLC is a separate presentation** — if they ask about it, say "I have a separate script for that" and switch to `MULTI-AGENT-SCRIPT.md`

Architecture walkthrough script — SupportIQ, Day 8 AI-Native Engineering Series